

Universidad Galileo	Guatemala 27 de septiembre 2026
Facultad: FISICC	Alumno: Luis Cordova
Curso: Microprocesadores	Carnet: 23012122
Sección: A	Hora de Laboratorio: 1:00 a 1:50
Auxiliar: Brandon Eduardo Mendoza Espinoza	Día de Laboratorio: Lunes

Laboratorio #6 Puertos Paralelos en C

Resumen:

Esta práctica consistió en programar, sobre la tarjeta Nucleo-G431KB, una aplicación en C que lee un joystick de cinco pulsadores y enciende un LED distinto según la dirección. Todo el manejo del GPIO se hizo escribiendo directo en los registros, sin apoyarse en bibliotecas.

La tabla de pines que trae el enunciado pertenece a la STM32F303 Evaluation Board. Como la Nucleo-G431KB solo tiene 32 pines y no saca los puertos C, D ni E, el circuito hubo que armarse desde cero con lo que sí está disponible: los pulsadores en el puerto A con pull-up interno, y los LEDs en el puerto B con resistencias de $220\ \Omega$.

El código emplea la macro MASK que pide el enunciado. Además de usarla, se verificó qué pasa realmente con ella: el preprocesador la cambia por texto antes de compilar y el código máquina resultante es idéntico al de escribir el desplazamiento a mano, aunque con un riesgo de precedencia que se ilustra con un ejemplo.

Objetivos:

- Configurar puertos GPIO a nivel de registro: habilitar el reloj en RCC, definir entradas y salidas en MODER y activar las resistencias internas en PUPDR.
- Diseñar y armar el circuito del joystick y los LEDs escogiendo pines que de verdad estén disponibles en la Nucleo-G431KB.
- Leer y escribir los puertos usando máscaras de bits y macros del preprocesador.

Teoría:

Los pines de propósito general del microcontrolador se agrupan en puertos de hasta dieciséis pines, nombrados A, B, C, etc. Se llaman puertos paralelos porque los dieciséis pines se leen y escriben al mismo tiempo, en un único registro de 32 bits. Cada puerto tiene sus registros propios: MODER para definir qué es cada pin, IDR para leer su estado, ODR para escribirlo y PUPDR para las resistencias internas [1][7].

MODER y PUPDR usan dos bits por pin, así que el pin n ocupa los bits $2n$ y $2n+1$; el pin 10, por ejemplo, se configura en los bits 20 y 21 [7]. En MODER, 00 es entrada, 01 salida, 10 función alterna y 11 analógico. En PUPDR, 00 deja el pin sin resistencia, 01 activa el pull-up y 10 el pull-down [1]. IDR y ODR, en cambio, usan un bit por pin.

Ningún puerto responde si antes no se le da reloj. El módulo RCC reparte la señal de reloj y cada periférico tiene su bit de habilitación; si no se activa, el puerto sigue apagado y las escrituras no hacen nada [3]. En el STM32G4 el GPIO cuelga del bus AHB2, y su reloj se habilita en `RCC->AHB2ENR` [7].

Como los dieciséis pines comparten el mismo registro, no se puede tocar uno sin afectar a los demás. Por eso se usan máscaras, que son valores con un uno en las posiciones que interesan y ceros en el resto; es común definirlos para las operaciones frecuentes [4]. Un OR con la máscara pone esos bits en uno, un AND con su complemento los pone en cero, y un AND a secas los aísla para leerlos.

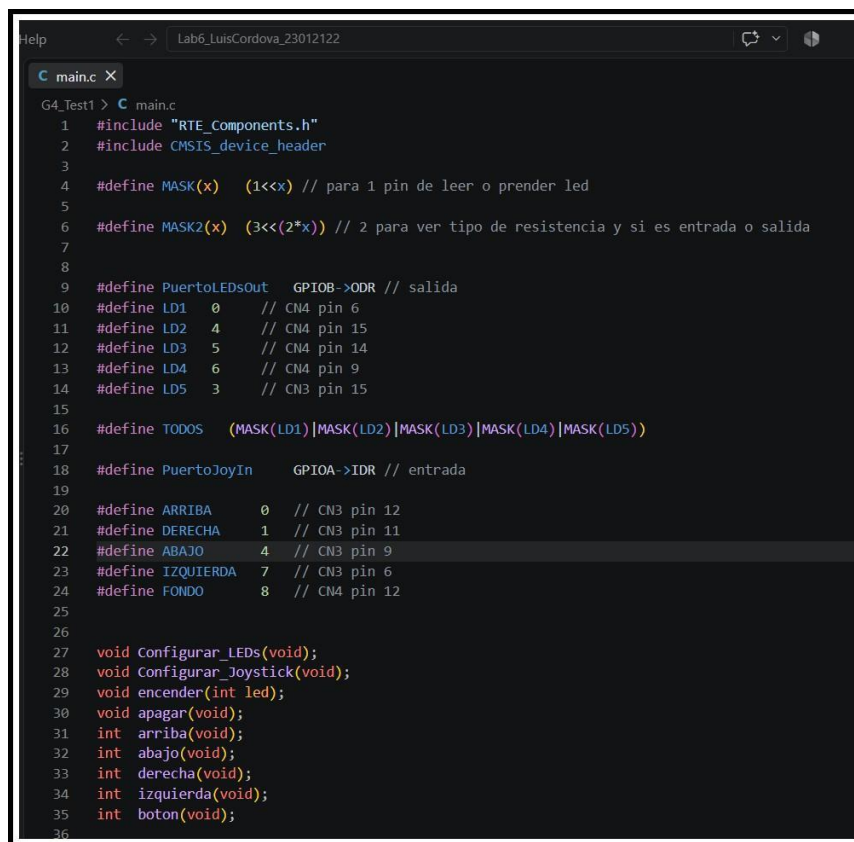
Una directiva `#define` con parámetros crea lo que se llama una macro tipo función: su uso se parece al de una función, pero el preprocesador la reemplaza por su texto antes de que el compilador vea el código [5]. Por eso, si los argumentos no van entre paréntesis, las reglas de precedencia de C pueden hacer que la expresión calcule algo distinto de lo esperado [6].

Un pin de entrada sin conectar queda flotando y lee valores al azar. La resistencia de pull-up lo mantiene en 1 mientras nadie lo toque; si el pulsador va entre el pin y tierra, al presionarlo el pin cae a 0. O sea que con pull-up la lógica queda invertida: presionado es 0 y suelto es 1.

Por último, los nombres impresos en la Nucleo-G431KB, como A0 o D3, son la nomenclatura de Arduino que ST usa para que la tarjeta sea compatible con sus accesorios. No son los puertos del microcontrolador: A3 es PA4 y D9 es PA8, del puerto A aunque su nombre empiece con D [2].

Resultados prácticos:

A continuación, se presentan las capturas obtenidas durante el desarrollo y la ejecución de la práctica.



```
Help  <  >  Lab6_LuisCordova_23012122
C main.c X
G4_Test1 > C main.c
1  #include "RTE_Components.h"
2  #include CMSIS_device_header
3
4  #define MASK(x)  (1<<x) // para 1 pin de leer o prender led
5
6  #define MASK2(x)  (3<<(2*x)) // 2 para ver tipo de resistencia y si es entrada o salida
7
8
9  #define PuertoLEDsOut  GPIOB->ODR // salida
10 #define LD1  0  // CN4 pin 6
11 #define LD2  4  // CN4 pin 15
12 #define LD3  5  // CN4 pin 14
13 #define LD4  6  // CN4 pin 9
14 #define LD5  3  // CN3 pin 15
15
16 #define TODOS  (MASK(LD1)|MASK(LD2)|MASK(LD3)|MASK(LD4)|MASK(LD5))
17
18 #define PuertoJoyIn  GPIOA->IDR // entrada
19
20 #define ARRIBA  0  // CN3 pin 12
21 #define DERECHA  1  // CN3 pin 11
22 #define ABAJO  4  // CN3 pin 9
23 #define IZQUIERDA  7  // CN3 pin 6
24 #define FONDO  8  // CN4 pin 12
25
26
27 void Configurar_LEDs(void);
28 void Configurar_Joystick(void);
29 void encender(int led);
30 void apagar(void);
31 int arriba(void);
32 int abajo(void);
33 int derecha(void);
34 int izquierda(void);
35 int boton(void);
36
```

Ilustración 1- macros MASK y MASK2, definiciones de los pines y prototipos de las funciones.

```

Help  <  >  Lab6_LuisCordova_23012122

C main.c X
G4_Test1 > C main.c
~
37  int main(void){
38
39      Configurar_LEDs();
40      Configurar_Joystick();
41
42      while(1){
43          if(arriba())      encender(LD1);
44          if(abajo())      encender(LD3);
45          if(derecha())    encender(LD2);
46          if(izquierda())  encender(LD4);
47          if(boton())      encender(LD5);
48
49          else              apagar();
50      }
51  }
52
53  void Configurar_LEDs(void){
54
55      RCC->AHB2ENR |= RCC_AHB2ENR_GPIOBEN; // activar reloj B
56      // limpiamos 00
57      GPIOB->MODER &= ~MASK2(LD1);
58      GPIOB->MODER &= ~MASK2(LD2);
59      GPIOB->MODER &= ~MASK2(LD3);
60      GPIOB->MODER &= ~MASK2(LD4);
61      GPIOB->MODER &= ~MASK2(LD5);
62      // poner como salida 01
63      GPIOB->MODER |= MASK(2*LD1);
64      GPIOB->MODER |= MASK(2*LD2);
65      GPIOB->MODER |= MASK(2*LD3);
66      GPIOB->MODER |= MASK(2*LD4);
67      GPIOB->MODER |= MASK(2*LD5);
68
69      // todos apagados al empezar
70      apagar();
71  }
72

```

Ilustración 2- lazo principal y función Configurar_LEDs.

```

Help  ← → Lab6_LuisCordova_23012122

C main.c
G4_Test1 > C main.c
73 void Configurar_Joystick(void){
74
75     RCC->AHB2ENR |= RCC_AHB2ENR_GPIOAEN;    // reloj A
76
77     // entrada 00
78     GPIOA->MODER &= ~MASK2(ARRIBA);
79     GPIOA->MODER &= ~MASK2(DERECHA);
80     GPIOA->MODER &= ~MASK2(ABAJO);
81     GPIOA->MODER &= ~MASK2(IZQUIERDA);
82     GPIOA->MODER &= ~MASK2(FONDO);
83
84     // limpiar 00
85     GPIOA->PUPDR &= ~MASK2(ARRIBA);
86     GPIOA->PUPDR &= ~MASK2(DERECHA);
87     GPIOA->PUPDR &= ~MASK2(ABAJO);
88     GPIOA->PUPDR &= ~MASK2(IZQUIERDA);
89     GPIOA->PUPDR &= ~MASK2(FONDO);
90
91     // pull-up 01
92     GPIOA->PUPDR |= MASK(2*ARRIBA);
93     GPIOA->PUPDR |= MASK(2*DERECHA);
94     GPIOA->PUPDR |= MASK(2*ABAJO);
95     GPIOA->PUPDR |= MASK(2*IZQUIERDA);
96     GPIOA->PUPDR |= MASK(2*FONDO);
97 }
98
99 // encender solo 1
100 void encender(int led){
101     PuertoLEDsOut = (PuertoLEDsOut & ~TODOS) | MASK(led);
102 }
103 // apagar todos
104 void apagar(void){
105     PuertoLEDsOut &= ~TODOS;
106 }

```

Ilustración 3- función Configurar_Joystick y funciones encender y apagar.

```

111 int abajo(void){
112     return !(PuertoJoyIn & MASK(ABAJO));
113 }
114 int derecha(void){
115     return !(PuertoJoyIn & MASK(DERECHA));
116 }
117 int izquierda(void){
118     return !(PuertoJoyIn & MASK(IZQUIERDA));
119 }
120 int boton(void){
121     return !(PuertoJoyIn & MASK(FONDO));
122 }
123

```

Ilustración 4- funciones de lectura del joystick.

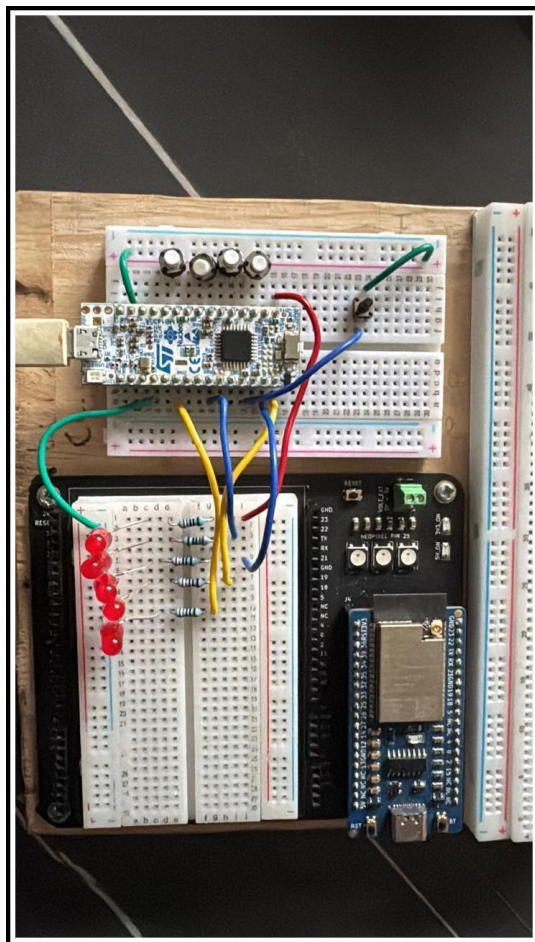


Ilustración 5- circuito armado: la tarjeta, los cinco pulsadores y los cinco LEDs con sus resistencias.

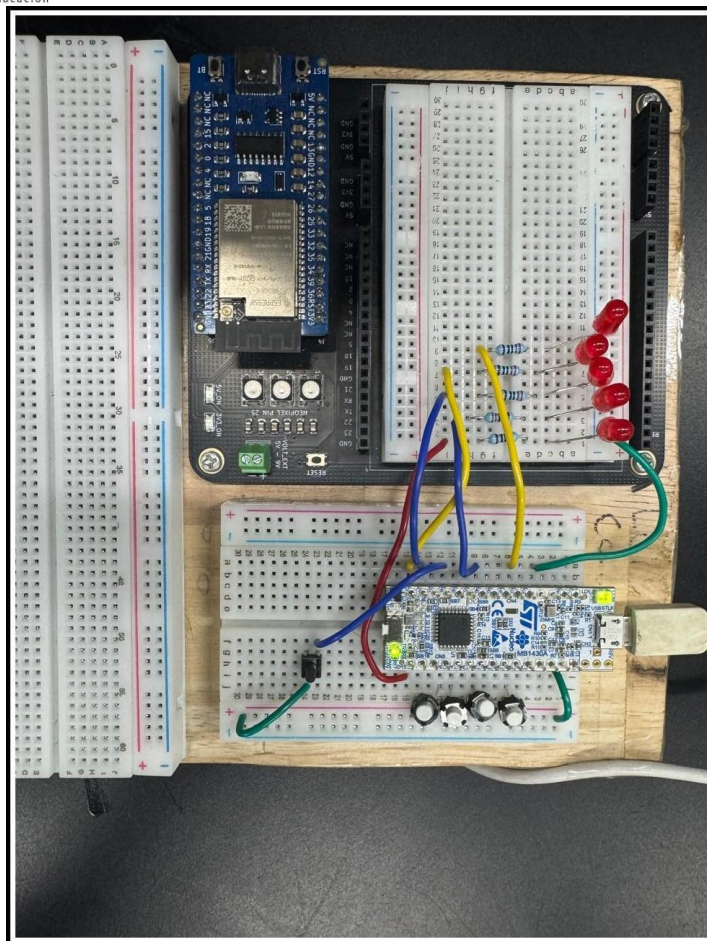


Ilustración 6- circuito conectado y en reposo, con los cinco LEDs apagados.

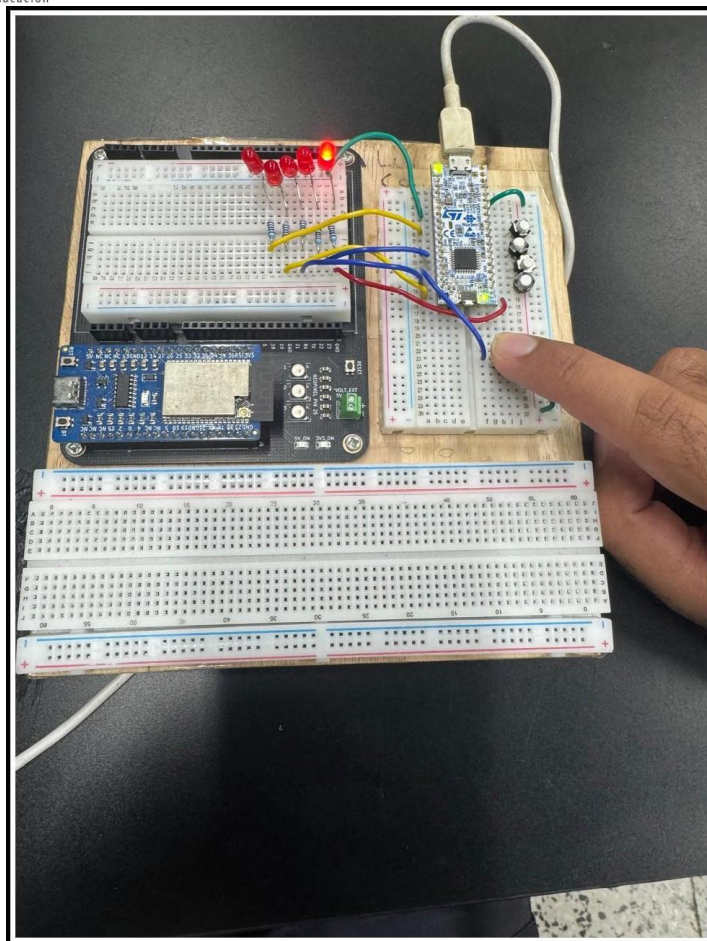


Ilustración 7- un pulsador presionado y su LED encendido.

Video de la práctica:

En el siguiente enlace se muestra el funcionamiento de la aplicación, recorriendo las cinco direcciones del joystick.

<https://youtube.com/shorts/v1XXXWkrt3I>

Resultados Teóricos:

Diseño del circuito

La tabla del paso 3 usa los pines PE7, PD6, PD5, PB5 y PC13 para el joystick, y PE8 a PE13 para los LEDs. Esos pines son de la STM32F303 Evaluation Board. En la Nucleo-G431KB los conectores solo sacan pines de los puertos A y B, más PF0 y PF1, que están compartidos con el oscilador [2]. El propio enunciado indica que cada estudiante debe seleccionar los pines disponibles en su tarjeta y decidir si usa pull-up o pull-down [7].

Se agruparon las entradas en el puerto A y las salidas en el puerto B, para que cada función de configuración trabaje sobre un solo puerto. Se evitaron PA2 y PA3, que son el puerto serial del ST-LINK; PF0 y PF1, del oscilador; y los pines cuya conexión al conector depende de puentes de soldadura. También se retiró el jumper HW1, que de fábrica une PA12 con tierra [2]. La asignación final quedó así:

Dirección	Pulsador	LED
Arriba	PA0 (A0)	LD1 · PB0 (D3)
Derecha	PA1 (A1)	LD2 · PB4 (D12)
Abajo	PA4 (A3)	LD3 · PB5 (D11)
Izquierda	PA7 (A6)	LD4 · PB6 (D6)
Selección	PA8 (D9)	LD5 · PB3 (D13)

Cada pulsador va del pin a tierra, sin resistencia externa, porque el pull-up lo pone el microcontrolador. Cada LED va del pin a una resistencia de 220 Ω , luego al LED y a tierra, así que enciende con un 1 lógico. Con un LED rojo, cuya caída es de unos 2 V, la corriente sale:

$$I = (3.3 \text{ V} - 2.0 \text{ V}) / 220 \Omega \approx 5.9 \text{ mA}$$

Queda por debajo de los 15 mA que fija el enunciado. Como el programa enciende un solo LED a la vez, esa es también la corriente total que entrega el puerto.

La macro MASK (paso 6)

El enunciado pide declarar la macro y explicar tres cosas: qué hace, qué tipo de función es y en qué se diferencia de lo que aparece al lado derecho de la asignación del paso anterior.

```
#define MASK(x) (1<<x)
```

Qué hace

Devuelve un valor con un único bit en uno, en la posición que recibe. Desplaza un 1 hacia la izquierda x lugares:

$\text{MASK}(0) = 1 \ll 0 = 0x0001$

$\text{MASK}(4) = 1 \ll 4 = 0x0010$

$\text{MASK}(14) = 1 \ll 14 = 0x4000$

Ese valor es una máscara: sirve para señalar un pin dentro de un registro. Con OR se enciende, con AND y el complemento se apaga, y con AND se lee.

Qué tipo de función es

No es una función de C, es una macro tipo función del preprocesador. Se escribe como si fuera una llamada, pero nunca se llama: antes de compilar, el preprocesador busca cada MASK(...) y la reemplaza por su texto. Pasando el ejemplo del paso 6 solo por el preprocesador se ve lo que realmente recibe el compilador:

Código escrito: `GPIOA->MODER |= MASK(2*7);`

Después del preprocesador: `GPIOA->MODER |= (1<<2*7);`

La macro desapareció. Por eso no tiene tipo de retorno, no revisa el tipo de su argumento, no ocupa memoria y no aparece en el ejecutable.

Diferencia con el lado derecho de la asignación

La primera diferencia a revisar es con escribir el desplazamiento directamente. Se compilaron las dos versiones para el Cortex-M4 y se comparó el código máquina:

Con MASK(2*7)	Con (1<<14)
<code>mov.w r1, #0x48000000</code>	<code>mov.w r1, #0x48000000</code>
<code>ldr r0, [r1]</code>	<code>ldr r0, [r1]</code>
<code>orr r0, r0, #0x4000</code>	<code>orr r0, r0, #0x4000</code>
<code>str r0, [r1]</code>	<code>str r0, [r1]</code>

Son idénticas. Como 2*7 es constante, el compilador resuelve el desplazamiento al compilar y deja el valor 0x4000 ya calculado. Usar MASK no cuesta ni una instrucción; la diferencia es de legibilidad, porque MASK(2*7) dice qué pin se está tocando y 0x4000 no.

La desventaja es que, al ser sustitución de texto, no protege la precedencia de operadores [6]. El operador OR tiene menos precedencia que el desplazamiento, así que:

$$\text{MASK}(x|1) \rightarrow (1 \ll x|1) \rightarrow (1 \ll x) | 1$$

Para x igual a 2 se esperaba $1 \ll 3$, que es 8, y se obtiene $4|1$, que es 5. La macro calcula otra cosa sin dar ningún error. Con $\text{MASK}(2*x+1)$ el resultado sí es correcto, pero solo porque la suma tiene más precedencia que el desplazamiento, y el compilador lo advierte en cada uso:

warning: suggest parentheses around '+' inside '<<' [-Wparentheses]

Hay un segundo detalle: el 1 de la macro es un int con signo, así que $\text{MASK}(31)$ desborda, y el compilador también lo advierte. Ninguno de los dos problemas ocurre en este programa, cuyo desplazamiento mayor es de 16, pero la forma segura sería $\#define \text{MASK}(x) (1U \ll (x))$.

La otra lectura posible es la constante que va en la asignación del paso 5, $\text{RCC} \rightarrow \text{AHB2ENR} |= \text{RCC_AHB2ENR_GPIOBEN}$. Esa constante también es una máscara, pero ya calculada para un solo bit y con nombre, definida por el fabricante en el archivo de cabecera del dispositivo; en el ejecutable se convierte en un orr con $0x2$, que es el bit 1. MASK es genérica y sirve para cualquier posición, pero hay que saber el número del bit; la constante del fabricante solo sirve para ese bit, pero su nombre documenta qué se está habilitando y evita equivocarse de posición.

Configuración de los puertos

Las dos funciones de configuración habilitan primero el reloj de su puerto y luego escriben los registros de dos bits. Para cada LED se hace en dos pasos:

```
GPIOB->MODER &= ~MASK2(LD1); // limpia los dos bits → 00
GPIOB->MODER |= MASK(2*LD1); // levanta el de abajo → 01 salida
```

El $2*$ convierte el número de pin en la posición de su primer bit en MODER. Limpiar antes hace falta porque $|=$ solo puede encender bits: si el bit de arriba viniera en 1, sin limpiar quedaría 11, que es modo analógico, y el LED no encendería. El ejemplo del paso 6 solo toca un bit y da por hecho que el otro está en cero; por eso se agregó una segunda macro que cubre los dos:

```
#define MASK2(x) (3 << (2*x))
```

El 3 es 11 en binario. MASK2 solo se usa negada, para limpiar; nunca para escribir, porque 11 no es un modo útil. Además evita escribir $\text{MASK}(2*\text{pin}+1)$, que es justo la expresión que provoca la advertencia de precedencia vista antes.

Para los pulsadores, MODER solo se limpia, porque entrada es 00. En PUPDR sí se limpian los dos bits y se levanta el de abajo, porque pull-up es 01. Así, cada pin queda con la misma configuración, aunque en una posición distinta del registro:

Pin	Bits en MODER y PUPDR	MODER	PUPDR
PA0	1 y 0	00	01
PA1	3 y 2	00	01
PA4	9 y 8	00	01
PA7	15 y 14	00	01
PA8	17 y 16	00	01

Lectura del joystick

Cada dirección se lee con una función de una línea:

```
int arriba(void){ return !(PuertoJoyIn & MASK(ARRIBA)); }
```

PuertoJoyIn es GPIOA->IDR, que entrega los dieciséis pines del puerto A a la vez. El AND con la máscara deja solo el bit de ese pulsador. Como hay pull-up, suelto da un valor distinto de cero y presionado da cero; el operador ! convierte el cero en 1 y cualquier otro valor en 0, así que la función devuelve verdadero justo cuando el pulsador está presionado. No es una variable: cada llamada vuelve a leer el registro, y por eso el programa detecta los cambios en cada vuelta del lazo.

Escritura de los LEDs

La constante TODOS reúne los cinco bits de los LEDs en una sola máscara, y con ella se escriben las dos funciones de salida:

```
#define TODOS (MASK(LD1)|MASK(LD2)|MASK(LD3)|MASK(LD4)|MASK(LD5))

void encender(int led){
    PuertoLEDsOut = (PuertoLEDsOut & ~TODOS) | MASK(led);
}

void apagar(void){
    PuertoLEDsOut &= ~TODOS;
}
```

encender() apaga los cinco LEDs, sin tocar los otros once pines del puerto B, y enciende el que recibe, todo en una sola escritura. apagar() solo apaga los cinco; Configurar_LEDs() la llama al final para que la aplicación arranque con todo apagado.

Lazo principal

El lazo evalúa las cinco direcciones con if independientes, en lugar de la cadena de else if que propone el paso 10:

```
while(1){
    if(arriba()) encender(LED1);
    if(abajo()) encender(LED3);
    if(derecha()) encender(LED2);
    if(izquierda()) encender(LED4);
    if(boton()) encender(LED5);

    else apagar();
}
```

Las cinco condiciones se revisan en todas las vueltas. Si un pulsador está presionado, encender() deja encendido solo su LED. Si se presionan dos a la vez, cada llamada borra la anterior y queda el último de la lista. Cuando no hay nada presionado, ninguna condición se cumple y el else final llama a apagar(), de modo que los cinco quedan apagados, como pide el paso 3 para el reposo.

Alcance del else

En C, un else pertenece únicamente al if que tiene inmediatamente antes. En este lazo, el else apagar() depende solo de if(boton()), no de las otras cuatro condiciones, así que también se ejecuta en cada vuelta en la que está presionado otro pulsador que no sea selección. En ese caso el LED se enciende al principio de la vuelta y se apaga al final. Se simuló el lazo midiendo qué parte de cada vuelta pasa encendido el LED mientras se mantiene presionado su pulsador:

Pulsador presionado	Parte de la vuelta con el LED encendido
Arriba	80 %
Abajo	60 %
Derecha	40 %
Izquierda	20 %
Selección	99 %

Como el lazo da millones de vueltas por segundo, el LED no se ve parpadear, pero lo esperable es que cada dirección brille con una intensidad distinta según su posición en la lista; el de selección, que está justo antes del else, no se ve afectado. Ni GCC ni Clang emiten advertencias, porque es C válido. Para que las cinco direcciones tuvieran el mismo brillo bastaría con volver a la cadena de else if del paso 10.

Verificación de la configuración

Antes de cargar el programa se ejecutaron las funciones de configuración sobre una copia de los registros en la computadora, para comprobar qué valores quedan escritos:

Registro	Resultado
RCC->AHB2ENR	0x00000003, puertos A y B con reloj
GPIOB->MODER	cinco pines de LED en 01, salida
GPIOA->MODER	cinco pines de pulsador en 00, entrada
GPIOA->PUPDR	cinco pines de pulsador en 01, pull-up

Los dos puertos quedan con reloj y cada pin en el modo y con la resistencia que le corresponden, sin alterar los pines que no se usan.

Conclusiones:

- La tabla del enunciado no se podía copiar tal cual: el diseño tiene que salir de los pines que la tarjeta realmente expone. En la Nucleo-G431KB solo quedan los puertos A y B, y aun dentro de ellos hubo que descartar los del ST-LINK, los del oscilador y el que venía unido a tierra por el jumper.
- MASK no es una función, es sustitución de texto. Genera exactamente el mismo código máquina que el desplazamiento escrito a mano, así que no cuesta nada, pero arrastra todos los problemas de precedencia: MASK(x|1) calcula otra cosa sin dar ningún error.
- El pull-up interno ahorra las cinco resistencias de los pulsadores a cambio de invertir la lógica, y configurarlo obliga a limpiar los dos bits antes de escribir, porque una operación |= solo puede encender bits y nunca apagarlos.

Bibliografía:

- [1] STMicroelectronics, «STM32G4 series advanced Arm-based 32-bit MCUs», Reference manual RM0440, Rev. 9, marzo de 2025, capítulo 9. [En línea]. Disponible en: https://www.st.com/resource/en/reference_manual/rm0440-stm32g4-series-advanced-armbased-32bit-mcus-stmicroelectronics.pdf
- [2] STMicroelectronics, «STM32G4 Nucleo-32 board (MB1430)», User manual UM2397, Rev. 2, septiembre de 2019. [En línea]. Disponible en: https://www.st.com/resource/en/user_manual/um2397-stm32g4-nucleo32-board-mb1430-stmicroelectronics.pdf
- [3] R. Cárdenas Rodríguez, J. Pagán Ortiz, A. Boscá Mojena, I. Martín Fernández y S. Esteban Romero, «Tutoriales sobre los fundamentos teóricos de sistemas basados en microcontrolador STM32», versión 2.0.1, Universidad Politécnica de Madrid, 2025. [En línea]. Disponible en: <https://oa.upm.es/88470/3/88470.pdf>
- [4] R. I. Zurita, «Dispositivos de E/S (periféricos)», apunte de la materia Programación de Sistemas Embebidos, Facultad de Informática, Universidad Nacional del Comahue, 2020. [En línea]. Disponible en: <https://se.fi.uncoma.edu.ar/pse/apuntes/perifericos-pse2020.pdf>
- [5] Free Software Foundation, «Function-like Macros», The C Preprocessor. [En línea]. Disponible en: <https://gcc.gnu.org/onlinedocs/cpp/Function-like-Macros.html>
- [6] Free Software Foundation, «Operator Precedence Problems», The C Preprocessor. [En línea]. Disponible en: <https://gcc.gnu.org/onlinedocs/cpp/Operator-Precedence-Problems.html>
- [7] J. Fajardo, «Práctica de Laboratorio #5: Puertos Paralelos en C», Microprocesadores, Universidad Galileo, septiembre de 2026.